Retrieval-Grounded MCQ Generation for a Generative-AI Course: A Multi-Agent Approach with Fine-Tuning, Critique, and Feedback

Anonymous Authors

***Abstract*—Generating high-quality multiple-choice questions (MCQs) with large language models is challenging because the questions must be clear, grounded, pedagogically useful, and supported by plausible distractors. This paper presents a retrieval-grounded multi-agent framework for MCQ generation in a Generative AI course. The system combines a fine-tuned Qwen3-4B generator, hybrid retrieval-augmented generation (RAG), and a LangGraph-based multi-agent workflow for planning, retrieval, generation, critique, distractor refinement, rebalancing, and student feedback. A lecture-grounded dataset of 2,111 MCQs was constructed from nine course lectures and annotated with question type, difficulty level, Bloom taxonomy level, tested concepts, evidence slide IDs, and answer rationales. The fine-tuned generator substantially outperformed the base model, increasing the overall generation score from 6.75 to 90.75. The multi-agent pipeline further improved grounding, reduced answer-length bias, and enhanced distractor quality. Results show that combining fine-tuning, RAG, and multi-agent critique can produce more reliable and course-aligned MCQs while also supporting personalized feedback and tutoring.**

***Index Terms*—Exam Generation, Large Language Models, Retrieval-Augmented Generation, Fine-Tuning, Multi-Agent Systems, LangGraph, Educational Assessment.**

I. Introduction

Multiple-choice questions are widely used in education because they allow instructors to assess students efficiently across different concepts and difficulty levels. However, writing high-quality MCQs is time-consuming and requires both subject knowledge and assessment design skills. A good MCQ should have a clear question stem, one correct answer, and distractors that are plausible enough to reveal misconceptions rather than being obviously incorrect. This becomes even more challenging in technical courses such as Generative AI and Large Language Models, where concepts are interconnected and often require more than simple memorization.

Large language models (LLMs) can generate fluent questions, but using them directly for exam generation introduces several risks. Generated questions may include unsupported information, weak distractors, answer-position bias, length bias, or difficulty labels that do not match the actual reasoning required. In a lecture-based course, another important requirement is grounding: each generated question should be traceable to the course material so that the assessment reflects what students were actually taught.

To address these challenges, this paper proposes a retrieval-grounded multi-agent framework for MCQ generation and student feedback in a Generative AI course. The system is built around three main ideas. First, a lecture-grounded MCQ dataset is constructed from nine course lectures and enriched with difficulty levels, Bloom taxonomy labels, tested concepts, evidence slide IDs, and explanatory rationales. Second, a Qwen3-4B base model is fine-tuned using LoRA to learn the required MCQ format and generation behavior. Third, a multi-agent workflow is used to improve the generated questions through planning, retrieval, grounding checks, distractor refinement, critique, rebalancing, and finalization.

The proposed system does not rely only on the fine-tuned generator. Instead, retrieval-augmented generation is used to retrieve relevant lecture evidence before generation, ensuring that the generated questions are supported by the course content. A curriculum planner controls the target concept, difficulty, Bloom level, and question type. A distractor specialist improves weak wrong answers, while critic and examiner agents check answer correctness, ambiguity, grounding, and common exam-design problems such as duplicate options and answer-length bias. The same lecture-grounded pipeline is also extended to a tutoring setting, where student answers are graded and explained using the relevant lecture evidence.

The main contributions of this paper are as follows:

- We construct a lecture-grounded MCQ dataset containing 2,111 questions from nine Generative AI and LLM lectures, annotated with question type, difficulty, Bloom level, concepts, evidence slides, and rationales.
- We fine-tune a Qwen3-4B base model using LoRA to generate structured MCQs aligned with the required educational format.
- We design a retrieval-grounded multi-agent architecture that combines planning, hybrid retrieval, on-topic checking, MCQ generation, distractor refinement, critique, rebalancing, and finalization.
- We extend the system beyond question generation by adding a student feedback and tutoring component

that explains student answers using retrieved lecture evidence.
- We evaluate the system against both a base model and a single-agent generator, showing that fine-tuning improves MCQ generation quality and that multi-agent orchestration improves grounding, distractor quality, and answer-length bias control.

II. Related Work

Automatic question generation is widely used in intelligent tutoring and assessment systems, yet generating high-quality multiple-choice questions (MCQs) remains challenging. A strong MCQ should have a clear stem, one correct and defensible answer, and distractors that are plausible enough to test students' misconceptions rather than being obviously wrong. Recent work by [1] shows that even strong LLMs such as GPT-4 can produce flawed MCQs, including questions that reveal the answer in the stem, overly long wording, weak distractors, and inconsistent reasoning. Their MCQG-SRefine framework addresses these issues using expert-designed prompts, self-critique, and revision. Our work follows the same motivation, but applies it to a lecture-based Generative AI course, where each generated question must be grounded in the provided lecture material and controlled by difficulty level and Bloom's taxonomy.

Our generation process is also inspired by the self-refinement paradigm proposed by [2], where an LLM improves its own output through feedback and revision. However, instead of relying only on one model to critique itself, our system uses a multi-agent workflow. The generator creates the initial MCQ, while separate agents check the question structure, answer correctness, distractor quality, ambiguity, grounding, and possible exam-design biases such as duplicate options or length bias.

To reduce hallucination and keep the questions faithful to the course content, our system uses retrieval-augmented generation (RAG) [3]. Rather than depending only on the model's internal knowledge, the system retrieves relevant lecture notes and slide evidence before generating or explaining an answer. This makes the generated MCQs more traceable and allows the tutoring component to refer students back to the relevant lecture content when they answer incorrectly.

Finally, we adapt a small open-source model using Low-Rank Adaptation (LoRA) [4]. LoRA freezes the original model weights and trains only a small number of additional low-rank parameters, making fine-tuning more efficient and feasible on limited hardware. In our system, LoRA is used to teach the model the required MCQ format and generation behavior, while RAG and the multi-agent workflow handle grounding, validation, and feedback.

Table I summarizes common limitations in LLM-based MCQ generation and how our system addresses them.

TABLE I
Limitations of LLM-based exam generation and proposed mitigation strategies.

Limitation	What goes wrong	Our mitigation
Answer-position bias	The correct answer may appear too often in specific positions, such as A or B, making the exam predictable [5].	The system balances answer positions during training and applies option shuffling during generation.
Length bias	The correct answer may be longer or more detailed than the distractors, allowing students to guess without understanding the concept [1].	A critic checks option length and a distractor-rebalancing step rewrites options to reduce this bias.
Weak distractors	Wrong options may be too obvious, unrelated to the concept, or unable to test real misconceptions [1].	A distractor-specialist agent generates plausible distractors, and an examiner agent checks their quality before finalizing the item.
Hallucination and weak grounding	The model may introduce facts that are not supported by the lecture material or cite irrelevant sources [1], [3].	RAG retrieves lecture-based evidence, and a groundedness check ensures that the question and feedback remain tied to the course content.
Difficulty and Bloom-level mismatch	The model may label a question as difficult or analytical even when it only tests simple recall.	A planner specifies the intended difficulty and Bloom level, while the critic checks whether the generated question matches the requested level.

III. Methodology

The section below outlies the implemented methodology covering the used dataset, system overiview, RAG, MCQ Fine-Tuning & Multi-Agent System.

A. Dataset Description

In this study, we constructed a lecture-grounded multiple-choice question (MCQ) dataset for evaluating and fine-tuning an automated MCQ generation system. The dataset was collected from three main sources. First, we used the course material, which consists of nine lectures covering core topics in generative artificial intelligence and large language models (LLMs), including LLM fundamentals, transformer architectures, pre-training and scaling laws, instruction fine-tuning, reinforcement learning from human feedback (RLHF), retrieval-augmented generation (RAG), and agentic AI workflows. Introductory, outline, divider, and reference-only slides were excluded to ensure that the dataset focuses only on examinable instructional content.

Second, each lecture was processed to generate structured lecture notes in JSON format using a predefined prompt. These notes captured the main concepts, definitions, mechanisms, examples, and slide-level evidence from each lecture. Third, an MCQ exam bank was assembled using lecture-derived questions and additional exam-style questions inspired by external educational resources, including NotebookLM and DeepLearning.AI course materials. All questions were manually or semi-automatically mapped back to the relevant lecture concepts and evidence slides to preserve grounding in the course content.

Each dataset instance represents one MCQ item and contains the question text, question type, answer options, correct answer, difficulty level, Bloom's taxonomy level, tested concepts, supporting evidence slide IDs, and explanatory rationales for the correct and incorrect options. The dataset includes three question formats: single-answer MCQs, true/false questions, and multiple-response questions. Difficulty was annotated using three levels: Beginner, Intermediate, and Difficult. Bloom levels were annotated using six categories: Remember, Understand, Apply, Analyze, Evaluate, and Create.

Table II summarizes the final dataset statistics, while Table III shows representative examples from the constructed MCQ dataset.

TABLE II
SUMMARY STATISTICS OF THE CONSTRUCTED MCQ DATASET.

Category	Sub-category	Count
Total	All MCQ Questions	2111
Lecture coverage	Lecture 1: Introduction to Generative AI and LLMs	178
	Lecture 2: Transformer Architecture	190
	Lecture 3: Pre-training and Scaling Laws	238
	Lecture 4: Fine-tuning, Instruction Tuning, and PEFT	269
	Lecture 5: Reinforcement Learning from Human Feedback	206
	Lecture 6: LLM Optimization and GenAI Applications	205
	Lecture 7: Retrieval Augmented Generation	290
	Lecture 8: Understanding Agentic AI	237
	Lecture 9: Agentic AI Part 2	298
Question type	Single-answer MCQ	1656
	True/False	276
	Multiple-response MCQ	179
Difficulty level	Beginner	664
	Intermediate	887
	Difficult	560
Bloom level	Remember	579
	Understand	491
	Apply	291
	Analyze	442
	Evaluate	196
	Create	112

B. System Overview

The proposed methodology is shown in Figure 1. The architecture is organized into three main stages: offline preparation, online exam generation, and online student feedback and tutoring. In the offline preparation stage, the lecture slides, per-slide notes, and the constructed MCQ bank are processed to build the training and retrieval resources. This includes bias-aware augmentation, fine-tuning a Qwen3-4B model using LoRA, indexing lecture notes for RAG using dense embeddings and BM25, and constructing a concept-indexed distractor pool from plausible wrong-answer rationales. The online exam generation subsystem then uses a curriculum planner, hybrid retriever, on-topic guard, fine-tuned MCQ generator, distractor specialist, critique loop, and finalization module to generate lecture-grounded MCQs with controlled difficulty, Bloom level, and distractor quality. Finally, the student feedback and tutoring subsystem grades student answers, retrieves relevant lecture evidence, provides grounded explanations, recommends supporting resources when needed, and produces progress reports summarizing student performance and misconceptions.

C. Prompt Design

Every agent is driven by a role-constrained system prompt that fixes its role, style, and output contract; the reasoning agents are additionally pinned to JSON output (`response_mime_type = application/json`). Three principles recur across all prompts: *grounding* (use only the provided lecture content), *structured output* (a fixed JSON schema rather than free text), and *explicit constraints* on the allowed values (difficulty, Bloom level, question type). The MCQ Generator prompt (Box III-C) is also the training-time system prompt, so the fine-tuned model and the prompted agents share one contract.

MCQ Generator (system).

You are an exam-question writer for a university course on Generative AI and Large Language Models. Given lecture content and a generation specification, produce exactly one multiple-choice question in valid JSON format.

- Use ONLY information present in the lecture content. Do not invent facts.
- Match the requested `question_type` exactly: `single_answer` (4 options A–D, exactly one correct), `true_false` (True/False), `multi_answer` (≥ 4 options, multiple correct).
- Wrong options must target real misconceptions a student might have.
- Cite slides in `evidence_slide_ids` as `L{lecture}-S{slide}`.
- Output ONLY the JSON object.

The remaining agents specialise this pattern. The **Curriculum Planner** converts a topic and target difficulty into a JSON spec (`concept`, `difficulty`, `bloom_level`, `question_type`, `retrieval_query`), constrained to the

TABLE III
REPRESENTATIVE SAMPLES FROM THE MCQ DATASET.

ID	Question Type	Question	Correct Answer	Difficulty	Bloom Level
Q001	Single-answer MCQ	Which statement best describes generative AI as presented in the lecture?	AI that learns patterns from data to create new content such as text, images, audio, or code.	Beginner	Remember
Q015	True/False	Language AI can often be used interchangeably with natural language processing (NLP).	True	Beginner	Remember
Q019	Multiple-response MCQ	Which are valid examples of generative AI use cases from the lecture? Select all that apply.	Text generation; image generation or enhancement; audio generation such as text-to-song.	Intermediate	Analyze

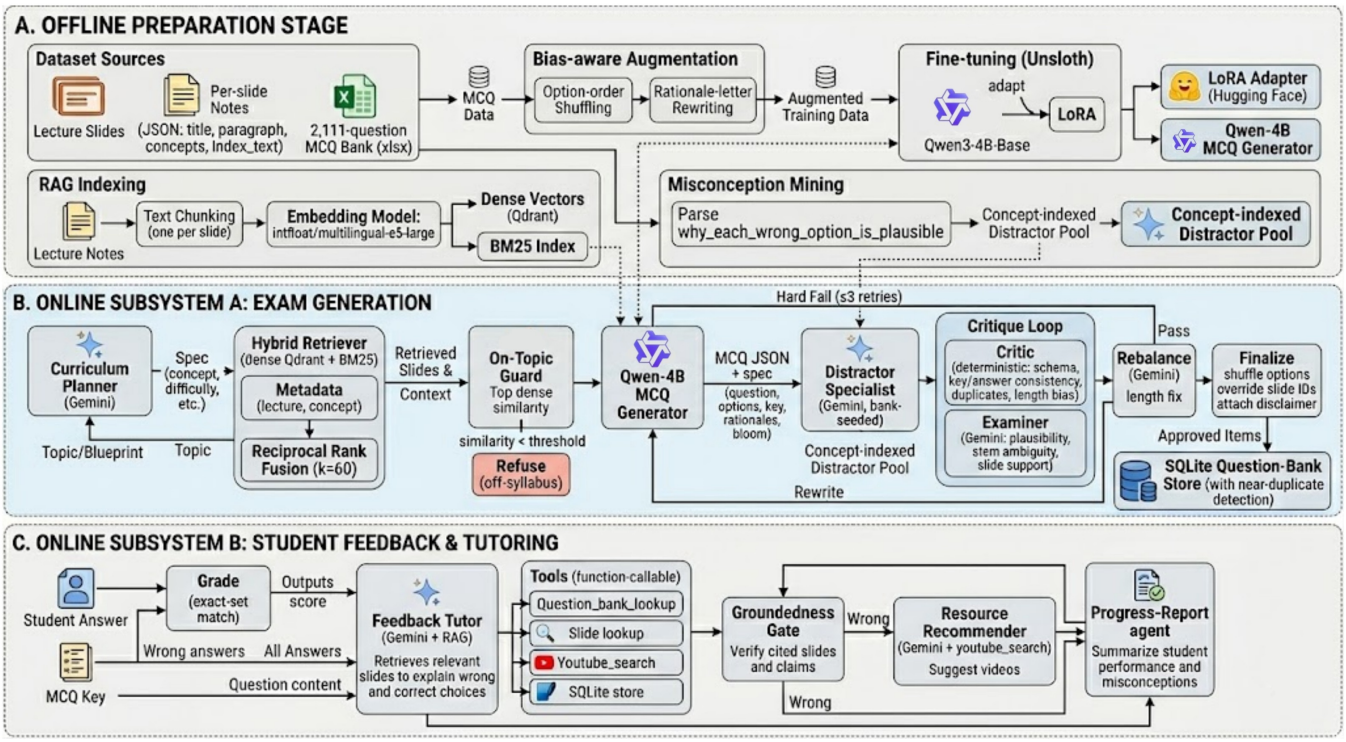

Fig. 1. Proposed multi-agent architecture for lecture-grounded MCQ generation and student feedback tutoring.

allowed enumerations; the **Exam Blueprint** planner instead composes a *balanced* set of such specs, spreading items across Bloom levels, difficulties, and types so no bucket dominates. The **Distractor Specialist** rewrites wrong options into plausible, misconception-based distractors that are explicitly required to be "similar in length and specificity to the correct answer," directly countering length bias. The **Feedback Tutor** explains a student's answer using only retrieved slide evidence and must cite slide IDs, and the **Resource Recommender** is instructed to call the `youtube_search` tool rather than invent links. The **Examiner** (Box III-C) is the quality-control prompt the deterministic Critic cannot replace: it reasons about meaning and returns a strict pass/fail verdict.

Examiner (system).

You are a strict exam-quality examiner for a Generative-AI course. You are given an MCQ and the lecture slide evidence it must be grounded in. Judge it on three axes and answer ONLY with JSON: {`distractors_plausible`, `stem_unambiguous`, `answer_supported_by_slides`, `issues`, `verdict`}.

- `distractors_plausible` is false if any wrong option is silly/obviously wrong (unrelated words, tautologies) rather than a real miscon-

ception.

- `stem_unambiguous` is false if more than one option could defensibly be correct, or the wording is vague.
- `answer_supported_by_slides` is false if the marked-correct option is not supported by the provided slide text.
- `verdict` is "fail" if ANY axis is false. Be conservative.

D. Retrieval-Augmented Generation

The retrieval corpus was built from the slide-level notes of the nine course lectures. Each slide was treated as one retrieval chunk, where the slide title, explanation, and key concepts were combined into a single passage. We also stored metadata for each chunk, including the lecture number, slide number, slide title, and key concepts. This produced a total of 664 lecture-based chunks, and the same slide identifiers were later used as evidence citations in the generated MCQs and student feedback. To support both semantic and keyword-based retrieval, we used a hybrid retrieval setup. Dense embeddings were stored in a local Qdrant vector database using cosine similarity, while a BM25 index was built over the same lecture chunks to capture exact keyword matches. Embedding Model used was `multilingual-e5-large` with normalized 1024-dimensional embeddings and the standard `query:/passage:` format. During retrieval, both the dense retriever and BM25 return candidate passages, which are then combined using Reciprocal Rank Fusion with equal weights. The final top 4–5 lecture chunks are passed to the generator as grounding evidence. Retrieval can also be filtered by lecture or concept, which allows the planner to generate questions from specific parts of the course. To prevent off-topic generation, an on-topic guard checks the top dense similarity score and rejects the request if it falls below 0.78. In the final system, RAG is used to ground MCQ generation, assign reliable evidence slide IDs, and generate lecture-based feedback when students answer incorrectly. In addition, a cosine similarity threshold of 0.93 is used to detect near-duplicate questions before adding them to the question bank.

E. Fine-Tuning the MCQ Generator

To adapt a general-purpose large language model for lecture-grounded multiple-choice question (MCQ) generation, a supervised fine-tuning (SFT) stage was performed using the `Qwen3-4B-Base` model. A base model was selected instead of an instruction-tuned variant to allow full control over the generated output structure and to align the model with the specific MCQ generation format required by the proposed educational framework.

To enable efficient training on limited computational resources, parameter-efficient fine-tuning was performed using Quantized Low-Rank Adaptation (QLoRA). The implementation was carried out using the Unsloth framework, which provides memory-efficient training and accelerated fine-tuning for large language models. The base model was loaded in 4-bit precision, significantly reducing memory consumption while preserving model performance.

LoRA adapters were attached to the transformer layers, allowing only a small subset of trainable parameters to be updated during training. This approach reduces both computational cost and memory requirements compared to full-model fine-tuning while maintaining adaptation capability for the target task.

The training dataset consisted of lecture-grounded MCQs represented in a structured format. Each training instance contained the question statement, answer options, correct answer, difficulty level, Bloom taxonomy level, tested concepts, evidence slide identifiers, and explanatory metadata. During preprocessing, the data were transformed into an instruction-following format that teaches the model to generate complete MCQs conforming to a predefined JSON schema.

To improve robustness and reduce positional bias, answer choices were randomized during preprocessing while preserving the corresponding correct-answer labels and explanations. This prevents the model from associating correctness with a fixed option position and encourages learning of the underlying concepts rather than answer placement patterns.

The resulting fine-tuned model was optimized to generate complete lecture-grounded MCQs, including pedagogical metadata such as difficulty levels, Bloom taxonomy categories, tested concepts, evidence references, and answer explanations. The generated output therefore follows a structured format suitable for direct integration into the downstream multi-agent assessment framework. To reduce bias in the training data, a bias-aware augmentation step was applied before fine-tuning. The order of answer options was shuffled while preserving the correct answer label, preventing the model from learning fixed answer-position patterns. In addition, rationales for correct and incorrect options were normalized to make the explanations more consistent across training examples. This preprocessing step helped the model learn the structure of high-quality MCQs rather than memorizing superficial option patterns.

F. Multi-Agent Architecture

To improve question quality, grounding, and pedagogical consistency, the proposed framework adopts a multi-agent architecture implemented using LangGraph. Rather than relying on a single model to perform planning, retrieval, generation, validation, and feedback, the workflow decomposes the task into specialized agents that collaborate through a shared state representation.

The architecture integrates a fine-tuned MCQ generator, a hybrid Retrieval-Augmented Generation (RAG)

module, deterministic validation components, and Gemini-based reasoning agents. Figure 1 illustrates the overall workflow.

1) Curriculum Planner and Exam Blueprint: The generation process begins with a Curriculum Planner agent. Given a target topic and difficulty level, the planner produces a structured specification containing:

- Concept to be assessed.
- Difficulty level.
- Bloom taxonomy level.
- Question type.
- Retrieval query.

The planner is implemented using Gemini and is guided by a dedicated planning prompt. The generated specification serves as the blueprint for downstream retrieval and question generation.

To support balanced assessment generation, an Exam Blueprint mechanism is also employed. The blueprint distributes questions across multiple difficulty levels, Bloom taxonomy categories, and question types to ensure comprehensive curriculum coverage.

2) Retriever and On-Topic Guard: To ensure that generated questions remain grounded in the course material, the system employs a hybrid Retrieval-Augmented Generation (RAG) module over the lecture notes.

The retrieval pipeline combines:

- Dense semantic retrieval using `intfloat/multilingual-e5-large` embeddings stored in Qdrant.
- Sparse lexical retrieval using BM25.
- Weighted Reciprocal Rank Fusion (RRF) for result aggregation.

Retrieved lecture slides are transformed into a structured evidence context that is passed to the generation agent. This grounding mechanism ensures that generated questions are supported by lecture content and can be traced back to specific slide identifiers. Before question generation, an On-Topic Guard agent evaluates whether the retrieval results provide sufficient evidence for the requested topic.

The guard computes a relevance score between the retrieval query and the retrieved lecture content. If the relevance score falls below a predefined threshold, the request is rejected and routed to a refusal node instead of allowing unsupported question generation.

This mechanism prevents the generation of off-syllabus or weakly grounded assessment items.

3) MCQ Generator: Question generation is performed using the fine-tuned `Qwen3-4B` model. The generator receives:

- The planning specification.
- Retrieved lecture evidence.
- Difficulty constraints.
- Bloom taxonomy requirements.
- Question-type requirements.

The generator produces a complete MCQ represented in a structured JSON format containing:

- Question statement.
- Answer options.
- Correct answer.
- Difficulty level.
- Bloom level.
- Tested concepts.
- Evidence slide identifiers.
- Explanations for both correct and incorrect options.

Generation is explicitly constrained to the retrieved lecture evidence to reduce hallucination and improve explainability.

4) Distractor Specialist: After question generation, a dedicated Distractor Specialist agent evaluates the quality of incorrect answer choices.

The specialist focuses on improving distractor plausibility while preserving answer correctness. Weak distractors are rewritten to better reflect common misconceptions and to increase the discriminative power of the assessment item.

This stage improves question quality by reducing trivial or obviously incorrect answer options.

5) Critic and Examiner: The framework employs two complementary validation components.

The **Critic** performs deterministic validation checks, including:

- JSON structure verification.
- Required-field validation.
- Question-type consistency checks.
- Correct-answer validation.
- Evidence-reference verification.

In parallel, a Gemini-based **Examiner** performs a higher-level pedagogical review. The examiner evaluates:

- Distractor plausibility.
- Question clarity.
- Ambiguity of the stem.
- Whether the cited evidence supports the answer.

Questions that fail either validation stage are returned for regeneration.

6) Rebalancing and Finalization: Following successful validation, a Rebalance agent performs final consistency adjustments. This stage ensures alignment between the generated question and the requested difficulty level, Bloom taxonomy category, and assessment specification.

The Finalize node then produces the final MCQ representation and prepares it for storage within the question bank.

7) Orchestration with LangGraph: All agents are orchestrated using LangGraph through a directed state graph.

The workflow follows the sequence:

$$\text{Planner} \rightarrow \text{Retriever} \rightarrow \text{Guard} \rightarrow \text{Generator} \rightarrow \text{Distractor Specialist} \rightarrow \text{Critic} \rightarrow \text{Rebalance} \rightarrow \text{Finalize}$$

Conditional routing enables iterative refinement whenever validation fails. In such cases, the workflow automatically returns to the generation stage and repeats the process until a valid question is produced or a predefined retry limit is reached.

This graph-based orchestration provides modularity, explainability, and quality control throughout the MCQ generation process.

G. Tools and Function Calling

The proposed system uses tool-based function calling to support retrieval, storage, feedback, and resource recommendation. During exam generation, the question-bank lookup tool checks whether a newly generated MCQ is too similar to previously approved questions before it is stored. This helps reduce near-duplicate items and improves the diversity of the final exam bank. The slide lookup tool retrieves the lecture evidence associated with a question, allowing both the generator and feedback tutor to cite the relevant course material. For student tutoring, the feedback agent uses the question bank and slide lookup tools to compare the student's answer with the correct answer and explain the mistake using grounded lecture evidence. When the system detects that a student needs additional support, the resource recommender calls a YouTube search tool to suggest external learning resources related to the weak concept. Approved questions and student interaction records are stored in a SQLite database, which allows the system to maintain a persistent question bank, track previous generations, and support progress reporting. Tool use is restricted to specific agents in the workflow. This design prevents the language model from inventing unsupported resources or citations and ensures that generated questions, explanations, and recommendations remain traceable

H. Student Feedback and Tutoring

Beyond question generation, the proposed framework provides an interactive tutoring component that supports formative assessment and personalized learning. After a student submits an answer, the tutoring workflow analyzes the response and generates feedback grounded in the lecture material.

A Feedback Tutor agent explains why the selected answer is correct or incorrect and highlights the concepts associated with the question. To support further learning, a Resource Recommender agent suggests supplementary educational resources related to the identified concept.

In addition, a Report Agent generates a structured performance summary that aggregates the student's interactions, identifies strengths and weaknesses, and highlights concepts requiring additional review. The generated report provides learners with actionable feedback while enabling instructors to monitor learning progress and concept mastery.

Together, these agents transform the generated MCQ bank from a static assessment resource into an adaptive tutoring environment that supports continuous learning and self-assessment.

IV. Results & Discussion

A. Experiemental Setup

We evaluate the two adaptation contributions separately, giving two baseline comparisons: base vs. fine-tuned generator (the effect of LoRA) and single- vs. multi-agent (the effect of the orchestration). The generator is `Qwen3-4B-Base` adapted with LoRA (r=16, 4-bit, Unsloth); the reasoning agents use Gemini (`gemini-3.1-flash-lite`); retrieval is dense (`multilingual-e5-large` over Qdrant) fused with BM25 via Reciprocal Rank Fusion over 664 slide chunks. We report automatic structural metrics from a deterministic scorer (schema completeness, option-count correctness, answer-key consistency, citation grounding duplicate-option rate, and the length-bias rate longest-is-correct, for which 0.25 is the unbiased ideal with four options) and **question-quality metrics** from anLLM-as-a-judge, GPT-5.5 Thinking, which rates eight criteria on a 0–10 scale. Using a judge from a different model than the system's own Gemini agents reduces self-preference bias.

B. Base Model vs. Fine-Tuned Generator

To evaluate the performance of fine-tuning, we compared the base Qwen3-4B model with the fine-tuned MCQ generator on a held-out test set of 60 samples. Each output was evaluated using 4 task-specific metrics, where each score represents the proportion of samples that satisfied the metric. A score of 1.00 therefore means that the model succeeded on all test samples for that metric, while 0.00 means that it failed on all samples.

The evaluation shows that the fine-tuned model performs better than the base model for MCQ generation. The base model can produce text in a readable format, but it fails to consistently follow the required MCQ structure, produce valid answer keys, or generate useful rationales. In contrast, the fine-tuned model learns the expected exam-generation behavior, including producing complete MCQ fields, generating the correct number of answer options, and maintaining consistency between the correct answer and the available choices.

The metrics were calculated as follows. *Output completeness* measures whether the generated question contains all required MCQ components, including the question, options, correct answer, difficulty, Bloom level, and explanation fields. *Option-count correctness* checks whether the number of answer choices matches the required question type, such as four options for a single-answer MCQ or two options for a true/false question. *Answer-key consistency* verifies that the marked correct answer actually exists among the generated options and does not contradict the provided answer choices. *Rationale quality* measures

TABLE IV
BASE MODEL VS. FINE-TUNED MCQ GENERATOR ON THE HELD-OUT TEST SET.

Metric	Base	Fine-tuned
Output completeness	0.00	**1.00**
Option-count correctness	0.27	**0.98**
Answer-key consistency	0.00	**0.90**
Rationale quality	0.00	**0.75**
Overall Score (0–100)	6.75	**90.75**

TABLE V
SINGLE- VS. MULTI-AGENT AUTOMATIC EVALUATION (n=100; LENGTH BIAS WAS CALCULATED ON THE 34 SINGLE-ANSWER ITEMS).

Metric	Single-agent	Multi-agent
Duplicate-option rate ↓	0.01	0.01
Longest-is-correct rate (ideal ≈ 0.25)	0.618	**0.000**
Grounded item rate ↑	0.92	**1.00**

whether the explanation justifies the correct answer and, when applicable, distinguishes it from the distractors.

As shown in Table IV, the fine-tuned generator outperforms the base model. The overall score increases from 6.75 for the base model to 90.75 for the fine-tuned model.Therefore, the fine-tuned model is the better generator and is more suitable for integration into the proposed multi-agent exam-generation pipeline.

C. Single- vs. Multi-Agent: Automatic Metrics

We evaluated whether the full multi-agent graph improves the output of the fine-tuned MCQ generator. In this comparison, the single-agent setting refers to using the fine-tuned generator directly, while the multi-agent setting refers to the complete pipeline, including retrieval, on-topic checking, distractor rewriting, critique, rebalancing, and finalization. The goal of this experiment was to measure whether the additional agents improve the practical quality of the generated questions beyond the raw fine-tuned model output. We evaluated on a sample size of 100 generated questions, as shown in Table V. Each metric was calculated as a proportion over the evaluated samples.

The *duplicate-option rate* measures how often a question contains repeated or nearly repeated answer choices. Lower values are better because duplicate options reduce exam quality and may reveal the answer. Both systems performed similarly on this metric, with a duplicate rate of 0.01.

The *longest-is-correct rate* measures whether the correct answer is frequently the longest option among the choices. This is important because students may exploit option length as a shortcut instead of reasoning about the content. For a balanced four-option MCQ, the ideal rate is approximately 0.25, since each option should have an equal chance of being the longest. The single-agent model produced a high rate of 0.618, meaning the correct answer was often noticeably longer than the distractors. The multi-agent system reduced this rate to 0.000, which removes the original length bias but slightly overcorrects in the opposite direction.

Finally, the *grounded item rate* measures whether the generated MCQ is supported by the retrieved lecture context. This metric was calculated by checking whether the question, correct answer, and explanation were consistent with the lecture material used by the system. The multi-agent system achieved perfect grounding (1.00), compared with 0.92 for the single-agent generator. This shows that the retrieval and critique components improve factual alignment with the course content.

Overall, the multi-agent system is better than the single-agent generator, it provides stronger grounding and greatly reduces answer-length bias.

D. Single- vs. Multi-Agent: LLM-as-a-Judge

In addition to the automatic metrics, we evaluated the quality of the generated questions using an LLM-as-a-judge. This evaluation compares the direct fine-tuned generator, referred to as the *single-agent* setting, with the complete *multi-agent* pipeline. The judge assigned a score from 0 to 10 for each criterion, where higher scores indicate better performance. Each criteria measures a different aspect of exam quality. *Question-type compliance* measures whether the generated item follows the requested format, such as single-answer MCQ, true/false, or multiple-response. *Answer-key correctness* checks whether the selected correct answer is actually valid and consistent with the options. *Lecture grounding/factuality* measures whether the question and its correct answer are supported by the lecture content rather than external or hallucinated knowledge. *Concept alignment* checks whether the question tests the intended concept. *Difficulty alignment* evaluates whether the actual reasoning required by the question matches the requested difficulty level. *Distractor quality* measures whether the wrong options are plausible, conceptually related, and based on realistic misconceptions rather than being obviously incorrect. *Clarity/ambiguity* checks whether the question is clearly written and has only one defensible answer. Finally, *workflow consistency* evaluates whether the generated item remains consistent with the overall pipeline specification, including the planned concept, retrieved lecture evidence, and final question requirements.

As shown in Table VI, both systems achieve high overall scores, which indicates that the fine-tuned generator already produces strong MCQ items. However, the multi-agent system improves the criteria that the additional agents are designed to address. In particular, distractor quality improves from 6.96 to 7.36, which is important because plausible distractors are central to high-quality MCQ construction. Lecture grounding also improves from 9.68 to 9.84, showing that the retrieval and critique com-

TABLE VI
SINGLE- VS. MULTI-AGENT LLM-AS-A-JUDGE EVALUATION SCORES ON A 0–10 SCALE.

Criterion	Single-agent	Multi-agent
Question-type compliance	**9.96**	9.90
Answer-key correctness	**9.95**	9.78
Lecture grounding / factuality	9.68	**9.84**
Concept alignment	**9.05**	8.97
Difficulty alignment	**7.84**	7.74
Distractor quality	6.96	**7.36**
Clarity / ambiguity	**8.65**	8.57
Workflow consistency	9.65	**9.81**
Overall	8.97	**9.00**

ponents help keep the generated questions more faithful to the lecture material. Workflow consistency improves from 9.65 to 9.81, indicating that the full graph better preserves the intended generation plan.

V. Limitations

The evaluation also has some limitations. The LLM-as-a-judge scores were produced by one automatic judge, so the absolute values should be interpreted carefully. They are most useful for comparing the single-agent and multi-agent settings rather than as final human-quality scores. In addition, the length-bias metric was calculated only on the single-answer MCQs in the 100-question run. The multi-agent system removed the original length bias, but it overcorrected slightly, since the correct answer was never the longest option. Ideally, this value should be closer to 0.25 for a four-option MCQ. Finally, difficulty alignment remained the weakest dimension for both systems, with scores around 7.8, showing that controlling the true reasoning difficulty of generated questions is still challenging. A future human-rating study with instructors or teaching assistants would provide stronger validation of the generated MCQs.

VI. Conclusion

This paper presented a retrieval-grounded multi-agent framework for generating and evaluating MCQs for a Generative AI course. The proposed system combines three complementary components: a fine-tuned Qwen3-4B MCQ generator, a hybrid RAG pipeline grounded in lecture slides, and a LangGraph-based multi-agent workflow for planning, generation, critique, distractor refinement, rebalancing, and student feedback. By grounding each question in lecture evidence and validating the generated items through multiple specialized agents, the system aims to produce MCQs that are not only structurally valid, but also pedagogically meaningful and aligned with the course content.

The experimental results show that fine-tuning is essential for adapting the base model to the required MCQ generation format. The fine-tuned generator achieved a much higher overall score than the base model, improving from 6.75 to 90.75 on the held-out test set. The multi-agent system further improved the generation process by increasing grounding from 0.92 to 1.00 and reducing answer-length bias from 0.618 to 0.000. In the LLM-as-a-judge evaluation, the multi-agent system achieved a slightly higher overall score than the single-agent setting, with noticeable gains in distractor quality, lecture grounding, and workflow consistency.

Overall, the results suggest that high-quality educational question generation benefits from combining model adaptation with retrieval and agent-based validation. Fine-tuning teaches the model the required MCQ structure, RAG keeps the output tied to the lecture material, and the multi-agent workflow improves quality control before the question is finalized. The tutoring component also shows how the generated MCQ bank can support formative learning by providing grounded explanations and progress reports for students.

There are still limitations that should be addressed in future work. The LLM-as-a-judge evaluation should be complemented with human evaluation by instructors or teaching assistants. Difficulty alignment also remains challenging, as both the single-agent and multi-agent systems sometimes struggle to match the intended reasoning level. In addition, the option-length rebalancing step should be tuned further so that it removes length bias without overcorrecting. Future work will focus on stronger human validation, improved difficulty calibration, larger-scale deployment, and deeper integration of student performance data into adaptive tutoring.

References

[1] Z. Yao, A. Parashar, H. Zhou, W. S. Jang, F. Ouyang, Z. Yang, and H. Yu, "MCQG-SRefine: Multiple choice question generation and evaluation with iterative self-critique, correction, and comparison feedback," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pp. 10728–10777, 2025.

[2] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[5] C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang, "Large language models are not robust multiple choice selectors," in *International Conference on Learning Representations (ICLR)*, 2024.